

Outline

- User authentication
 - Password authentication, salt
 - Challenge-response authentication protocols
 - Biometrics
 - Token-based authentication
- Authentication in distributed systems (multi service providers/domains)
 - Single sign-on, Microsoft Passport
 - Trusted Intermediaries

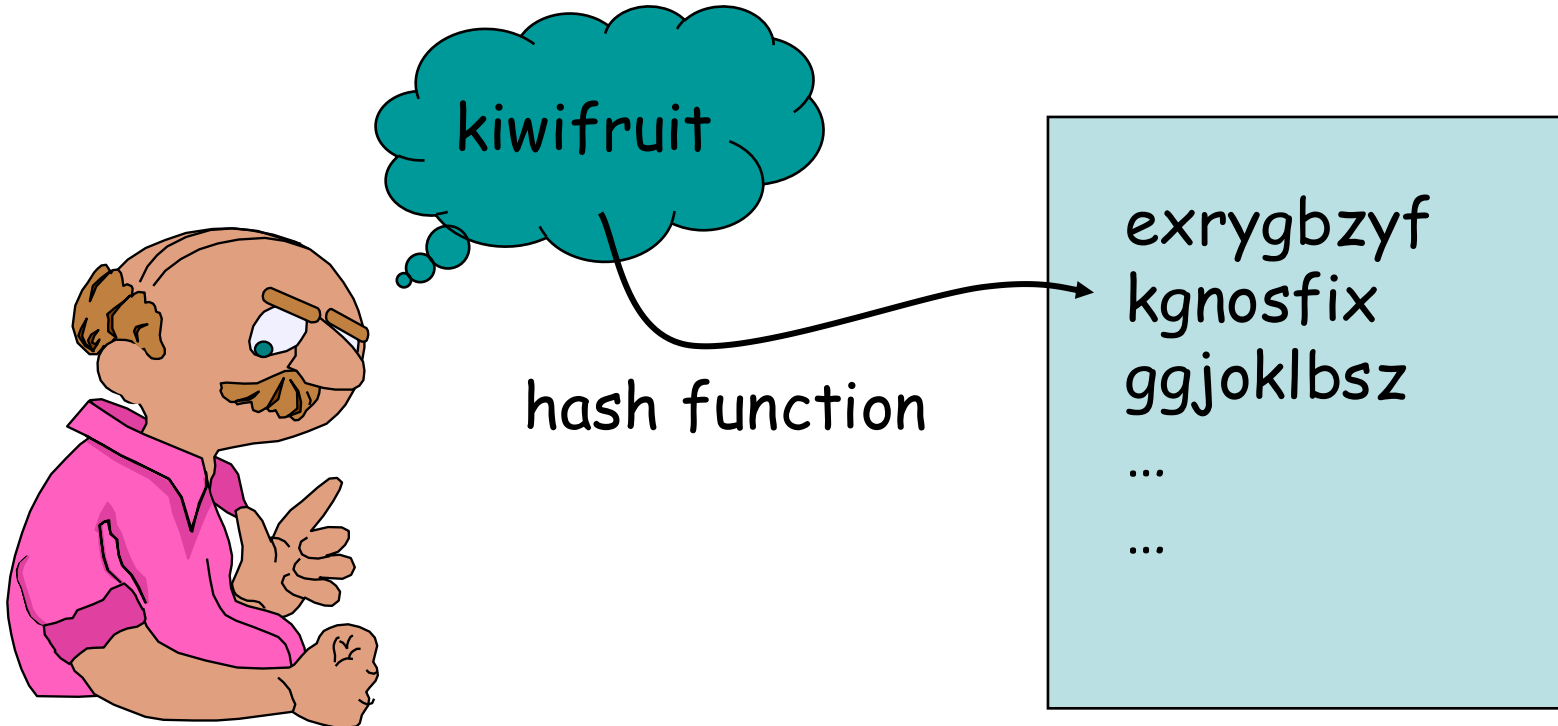
Password authentication

- Basic idea
 - User has a secret password
 - System checks password to authenticate user
- Issues
 - How is password stored?
 - How does system check password?
 - How easy is it to guess a password?
 - Difficult to keep password file secret, so best if it is hard to guess password even if you have the password file

Basic password scheme

User

Password file



Basic password scheme

- Hash function $h : \text{strings} \rightarrow \text{strings}$
 - Given $h(\text{password})$, hard to find password
 - No known algorithm better than trial and error
- User password stored as $h(\text{password})$
- When user enters password
 - System computes $h(\text{password})$
 - Compares with entry in password file
- No passwords stored on disk

Unix password system

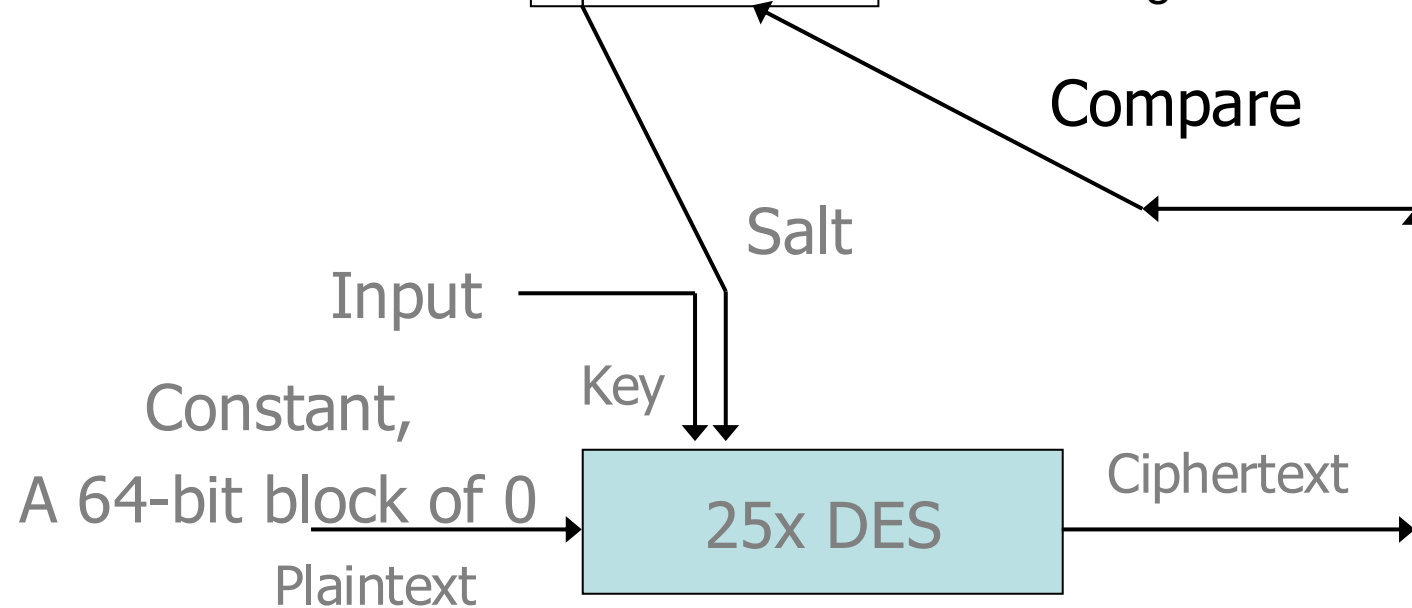
- Hash function is 25xDES
 - 25 rounds of DES-variant encryptions
- Password file is publicly readable
 - Other information in password file ...
- Any user can try "dictionary attack"
 - User looks at password file
 - Computes hash(word) for every word in dictionary
- "Salt" makes dictionary attack harder

R.H. Morris and K. Thompson, Password security: a case history, Communications of the ACM, November 1979

Salt

- Password line

walt:fURfuu4.4hY0U:129:129:Belgers:/home/walt:/bin/csh



When password is set, salt is chosen randomly
12-bit salt slows dictionary attack by factor of 2^{12}

Dictionary Attack - some numbers

- Typical password dictionary
 - 1,000,000 entries of common passwords
 - people's names, common pet names, and ordinary words.
 - Suppose you generate and analyze 10 guesses per second
 - This may be reasonable for a web site; offline is *much* faster
 - Dictionary attack in at most 100,000 seconds = 28 hours, or 14 hours on average
- If passwords were random
 - Assume six-character password
 - Upper- and lowercase letters, digits, 32 punctuation characters
 - 689,869,781,056 password combinations.
 - Exhaustive search requires 1,093 years on average

Outline

- User authentication
 - Password authentication, salt
 - Challenge-response authentication protocols
 - Biometrics
 - Token-based authentication
- Authentication in distributed systems (multi service providers/domains)
 - Single sign-on, Microsoft Passport
 - Trusted Intermediaries

Challenge-response Authentication

Goal: Bob wants Alice to “prove” her identity to him

Protocol ap1.0: Alice says “I am Alice”



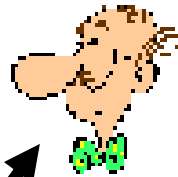
Failure scenario??



Authentication

Goal: Bob wants Alice to “prove” her identity to him

Protocol ap1.0: Alice says “I am Alice”

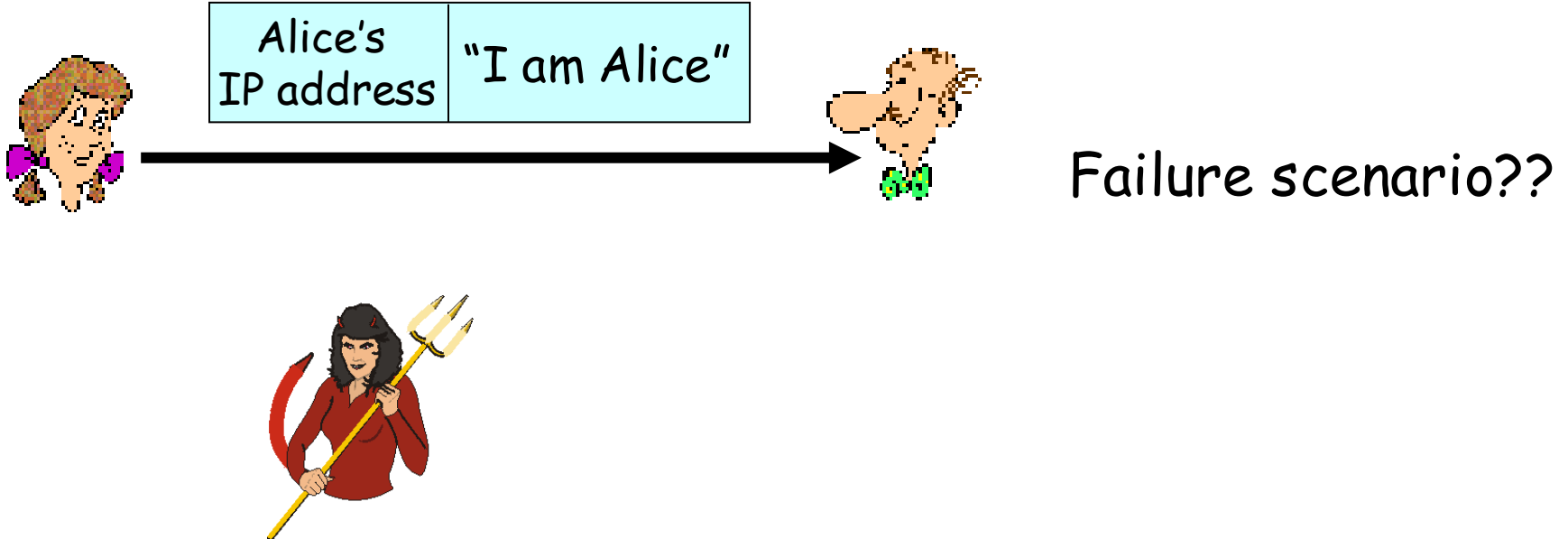


“I am Alice”

in a network,
Bob can not “see”
Alice, so Trudy simply
declares
herself to be Alice

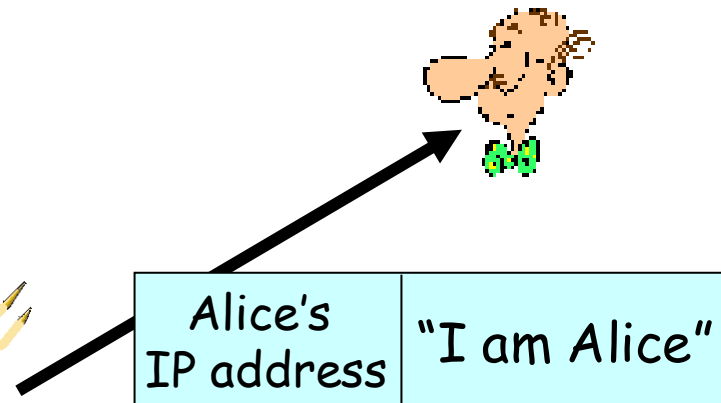
Authentication: another try

Protocol ap2.0: Alice says "I am Alice" in an IP packet containing her source IP address



Authentication: another try

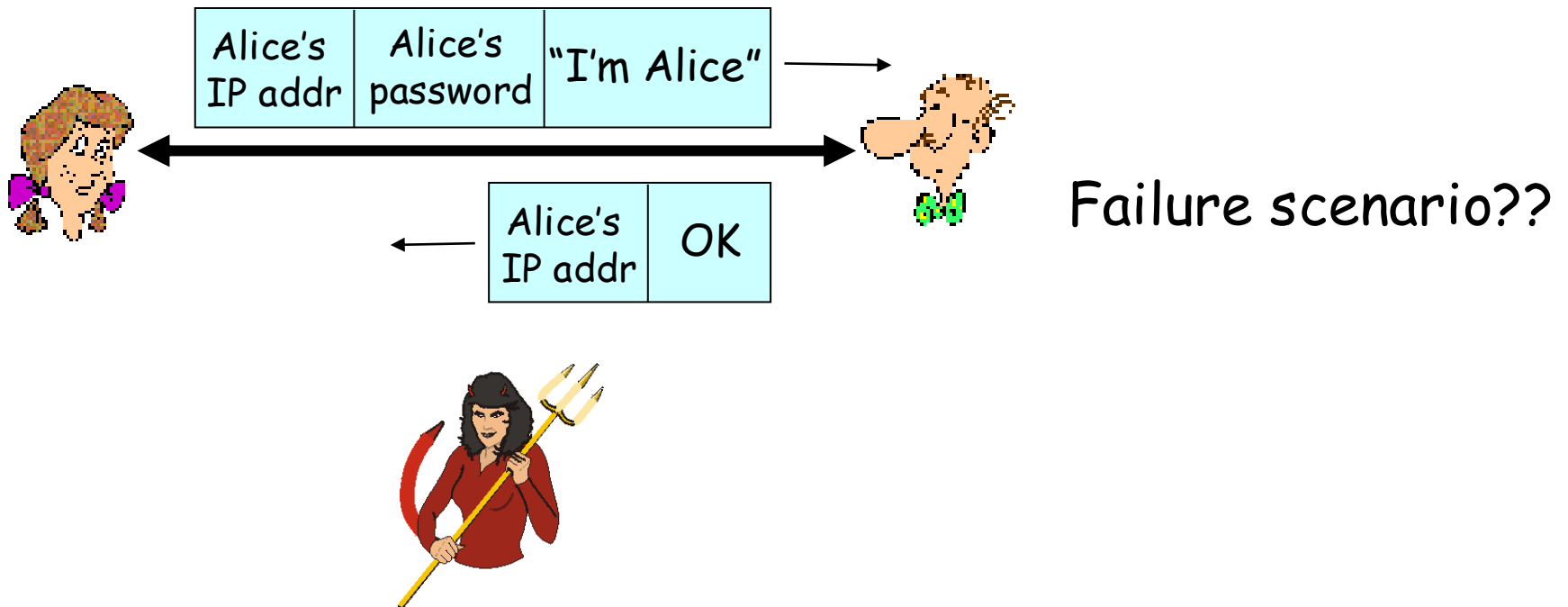
Protocol ap2.0: Alice says "I am Alice" in an IP packet containing her source IP address



Trudy can create
a packet
"spoofing"
Alice's address

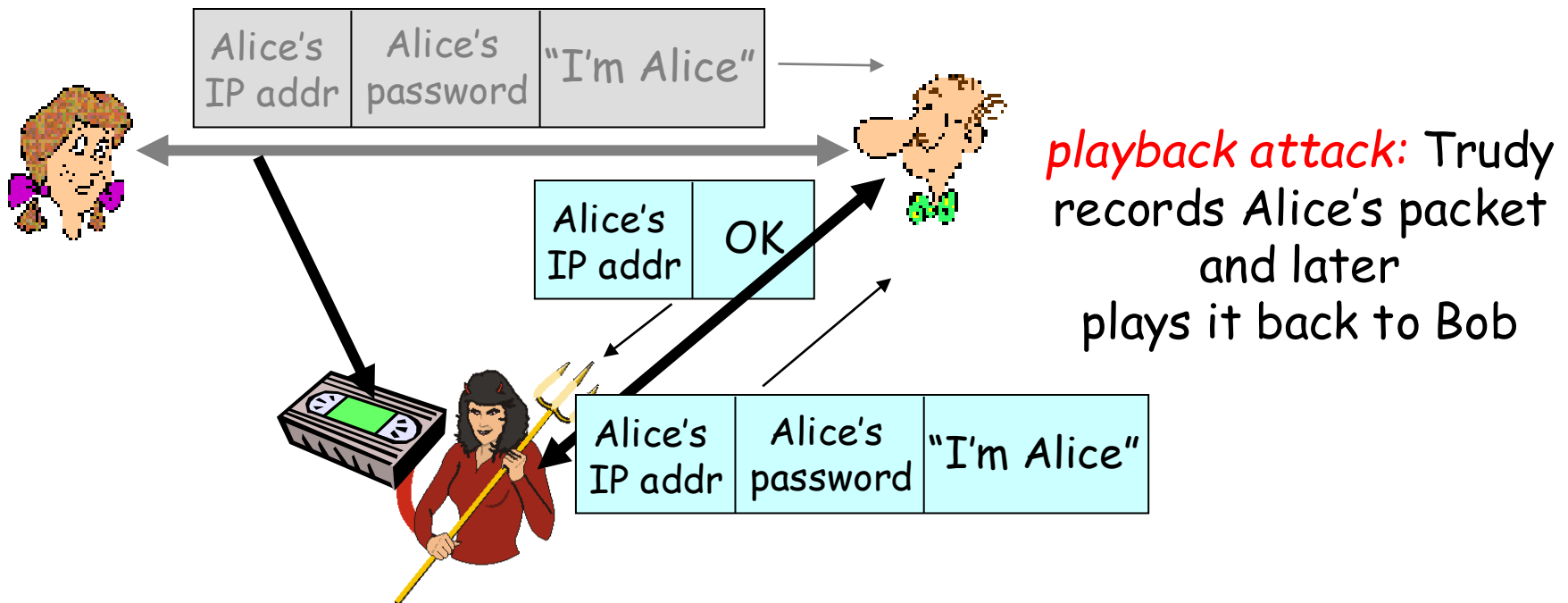
Authentication: another try

Protocol ap3.0: Alice says "I am Alice" and sends her secret password to "prove" it.



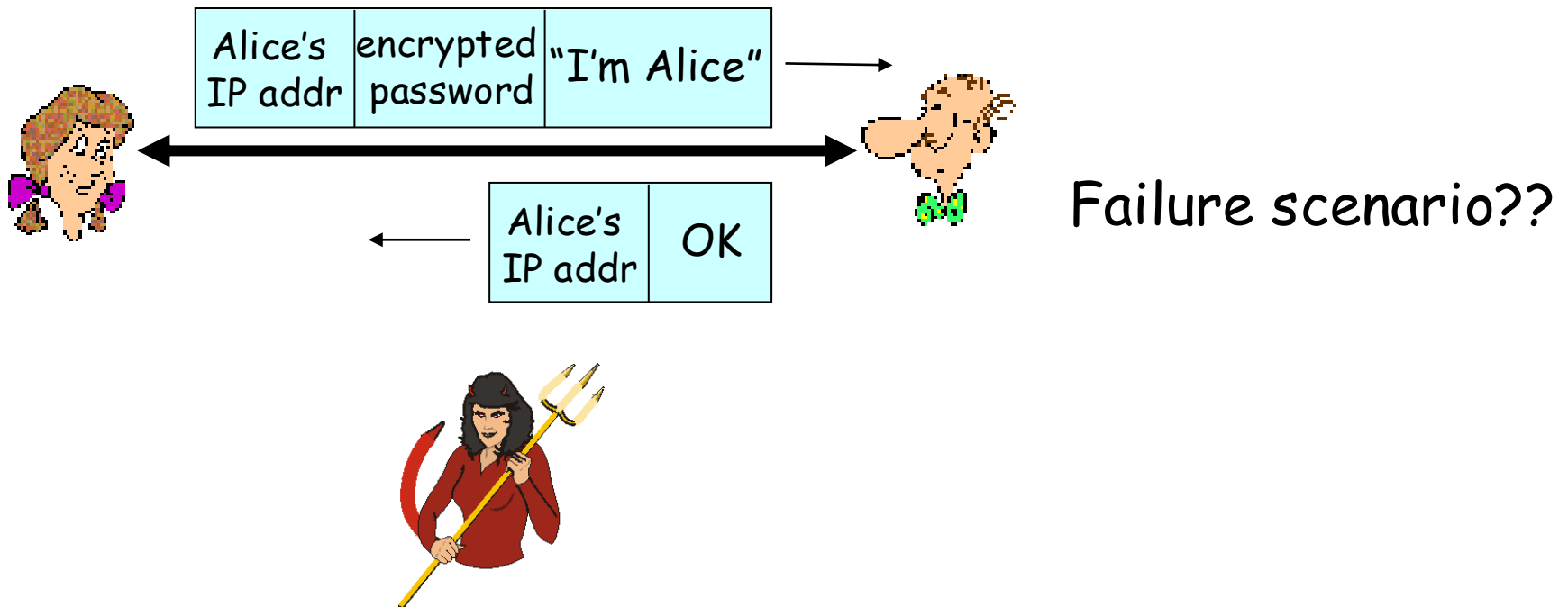
Authentication: another try

Protocol ap3.0: Alice says "I am Alice" and sends her secret password to "prove" it.



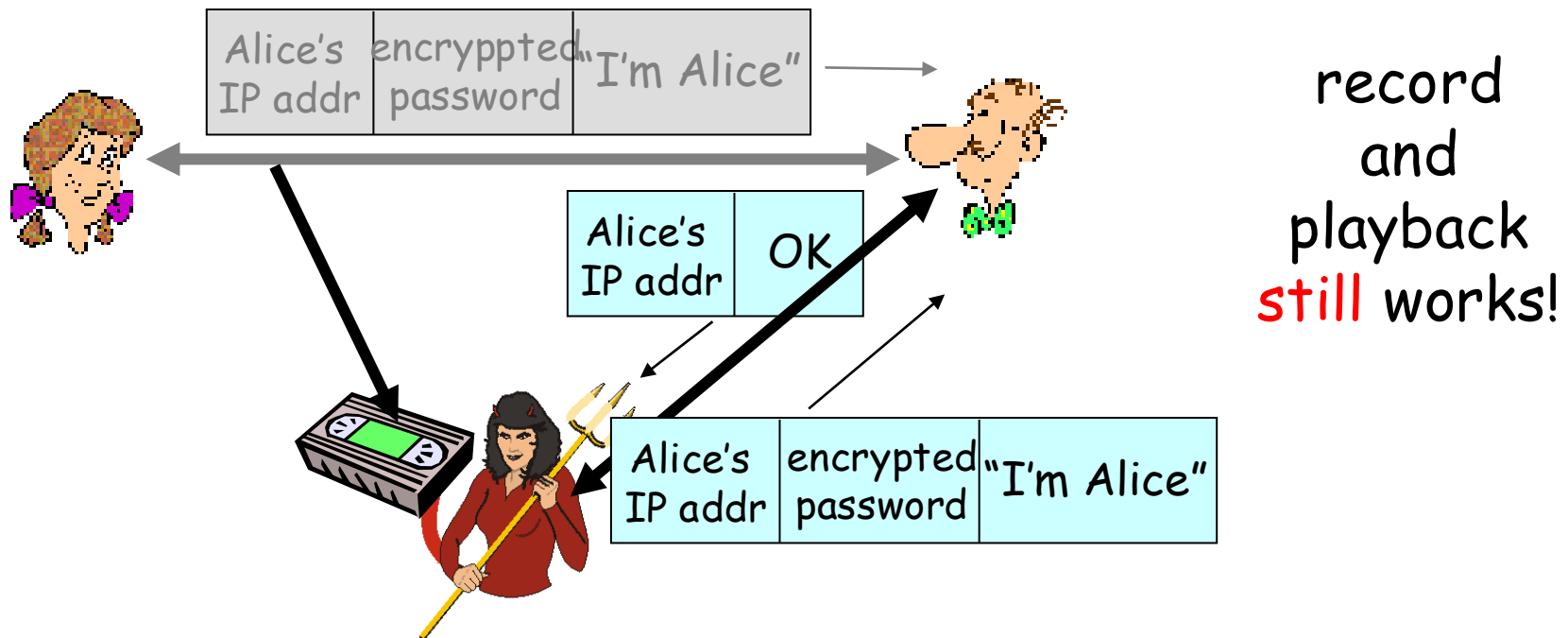
Authentication: yet another try

Protocol ap3.1: Alice says "I am Alice" and sends her *encrypted* secret password to "prove" it.



Authentication: another try

Protocol ap3.1: Alice says "I am Alice" and sends her *encrypted* secret password to "prove" it.

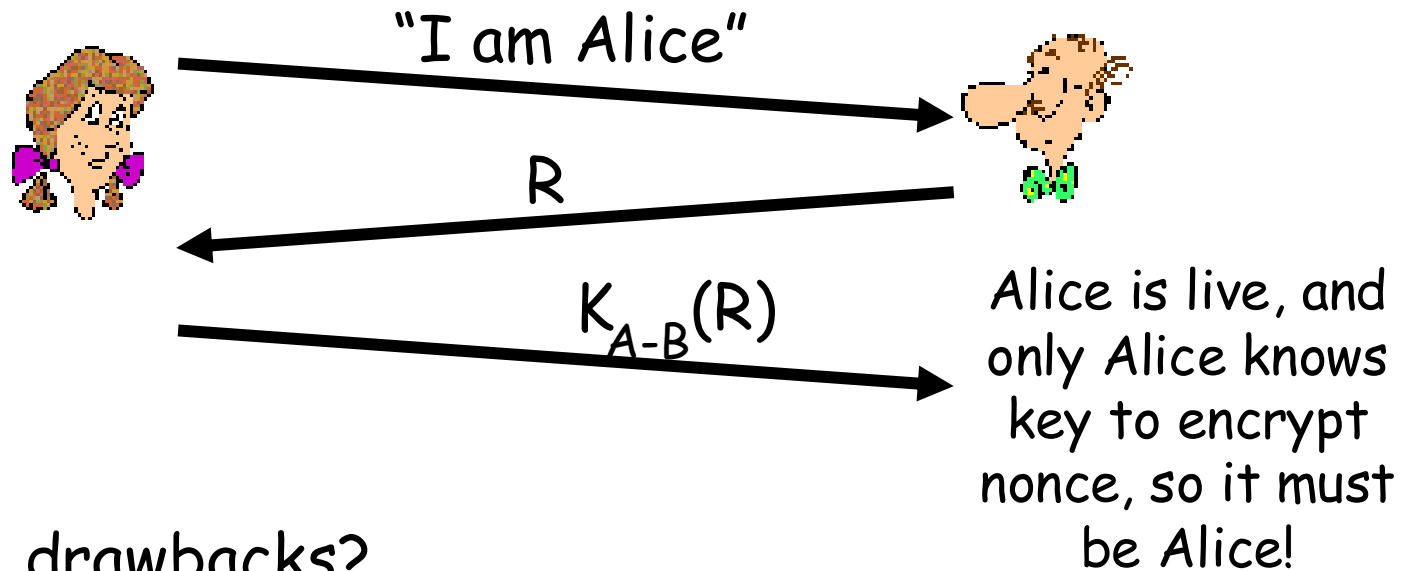


Authentication: yet another try

Goal: avoid playback attack

Nonce: number (R) used only *once -in-a-lifetime*

ap4.0: to prove Alice "live", Bob sends Alice **nonce**, R. Alice must return R, encrypted with shared secret key



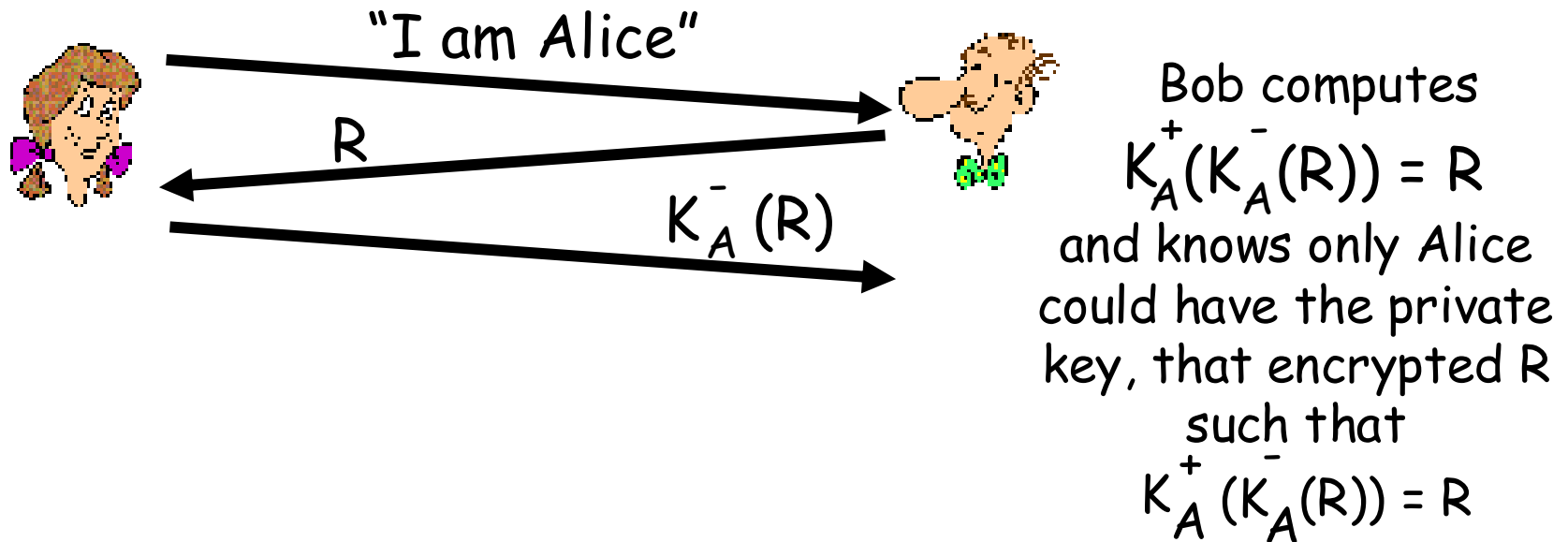
Failures, drawbacks?

Authentication: ap5.0

ap4.0 doesn't protect against server database reading

- can we authenticate using public key techniques?

ap5.0: use nonce, public key cryptography



Outline

- User authentication
 - Password authentication, salt
 - Challenge-response authentication protocols
 - Biometrics
 - Token-based authentication
- Authentication in distributed systems (multi service providers/domains)
 - Single sign-on, Microsoft Passport
 - Trusted Intermediaries

Biometrics



- Use a person's physical characteristics
 - fingerprint, voice, face, keyboard timing, ...
- Advantages
 - Cannot be disclosed, lost, forgotten
- Disadvantages
 - Cost, installation, maintenance
 - Reliability of comparison algorithms
 - False positive: Allow access to unauthorized person
 - False negative: Disallow access to authorized person
 - Privacy?
 - If forged, how do you revoke?



Biometrics

- Common uses
 - Specialized situations, physical security
 - Combine
 - Multiple biometrics
 - Biometric and PIN
 - Biometric and token



Token-based Authentication

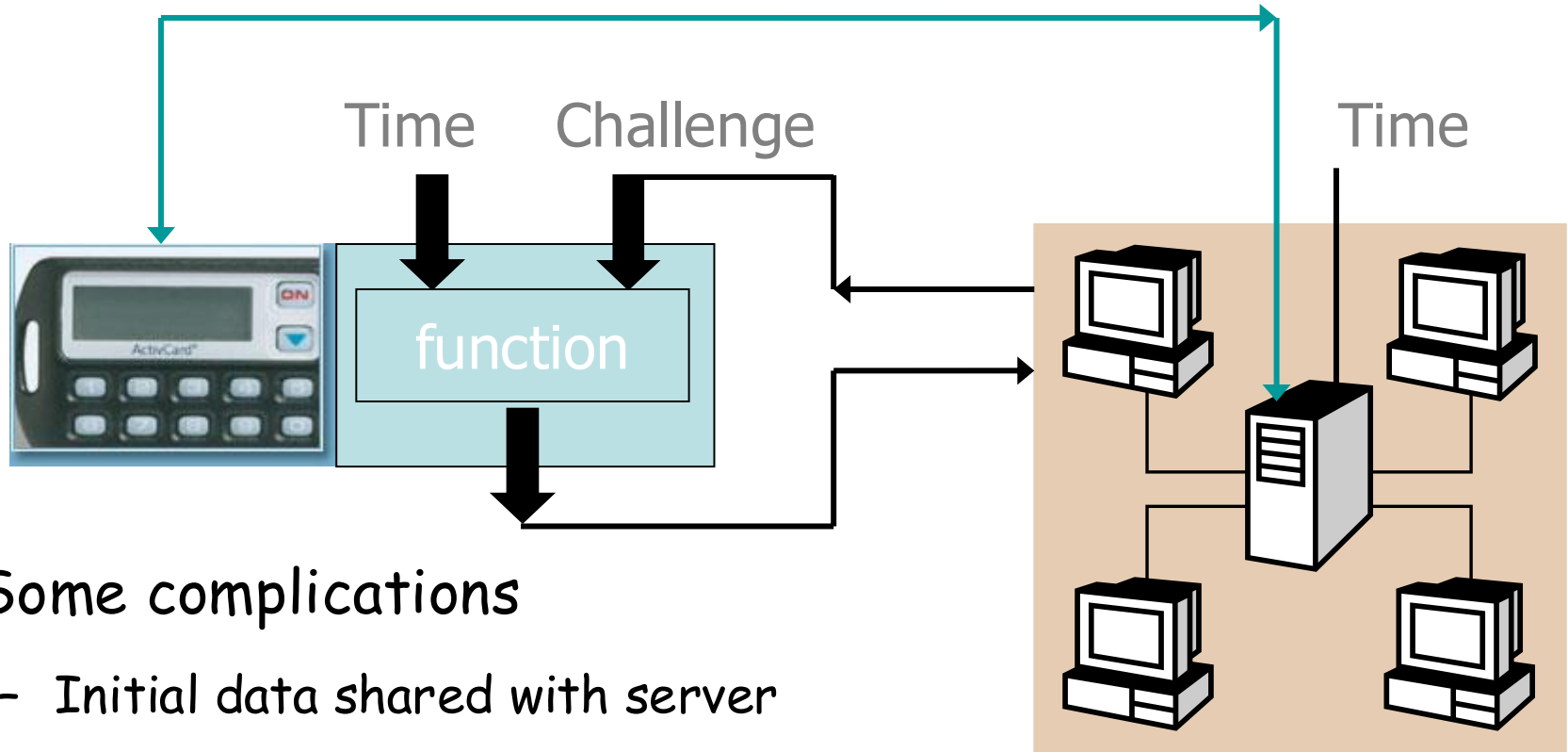
Smart Card



- With embedded CPU and memory
 - Carries conversation w/ a small card reader
- Various forms
 - PIN protected memory card
 - Enter PIN to get the password
 - Cryptographic challenge/response cards
 - A cryptographic key in memory
 - Computer create a random challenge
 - Enter PIN to encrypt/decrypt the challenge w/ the card
 - Cryptographic Calculator (readerless smart card)
 - Simulating a smartcard: user enter the encrypted result

Smart Card Example

Initial data



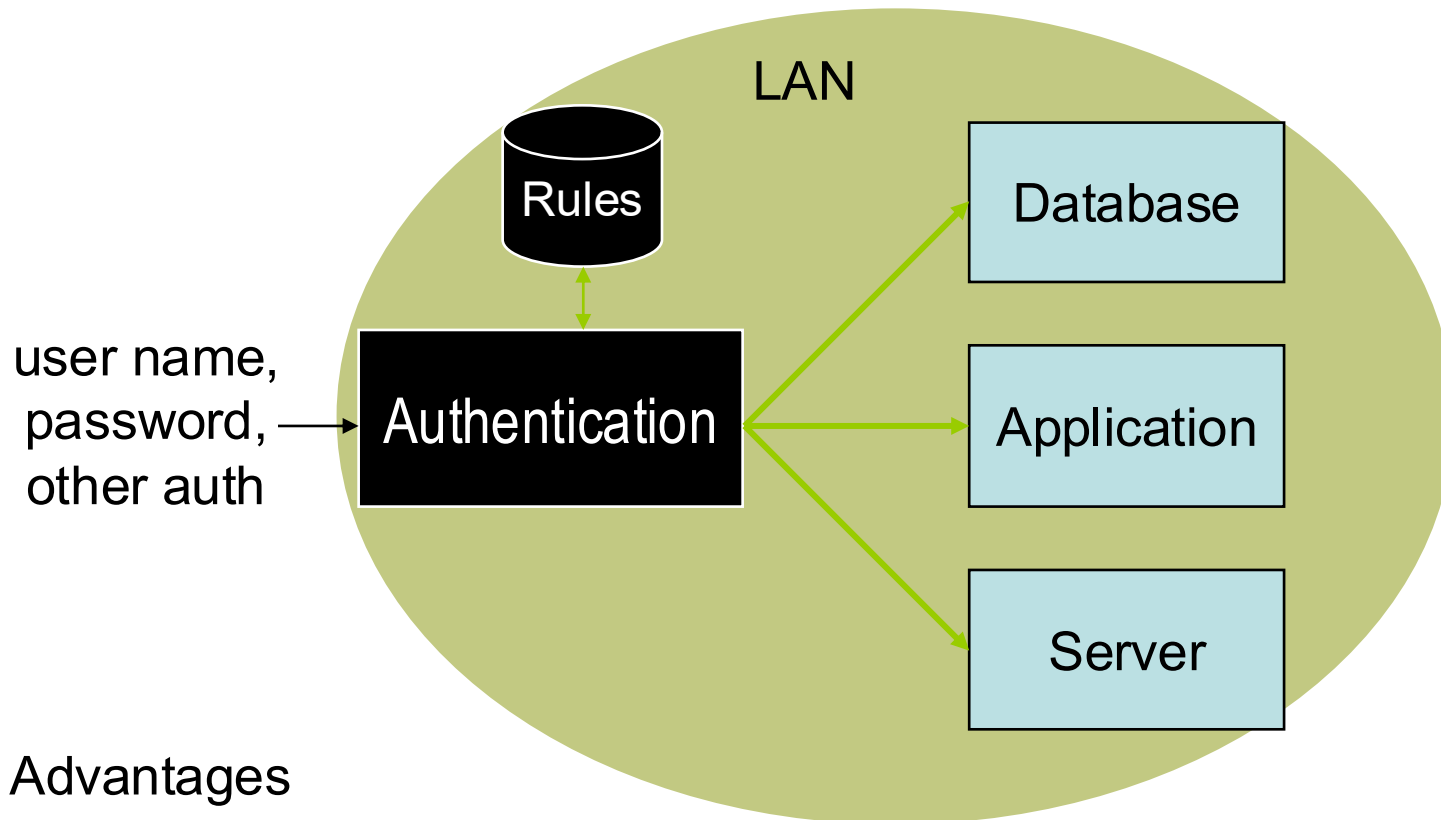
- Some complications
 - Initial data shared with server
 - Need to set this up securely
 - Shared database for many sites
 - Clock skew

Outline

- User authentication
 - Password authentication, salt
 - Challenge-Response
 - Biometrics
 - Token-based authentication
- Authentication in distributed systems
 - Single sign-on, Microsoft Passport
 - Trusted Intermediaries

Single sign-on systems

e.g. Securant, Netegrity, Oblix



- Advantages
 - User signs on once
 - No need for authentication at multiple sites, applications
 - Can set central authorization policy for the enterprise

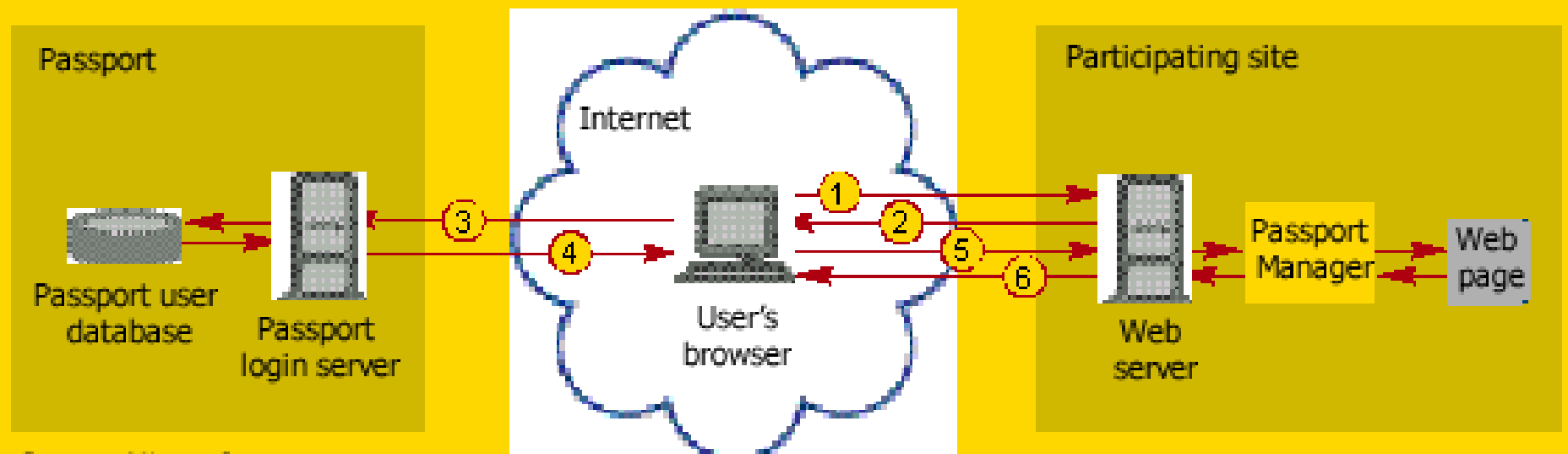
Microsoft Passport

- Launched 1999
 - Claim > 200 million accounts in 2002
 - Over 3.5 billion authentications each month
- Log in to many websites using one account
 - Used by MS services Hotmail, MSN Messenger or MSN subscriptions; also Radio Shack, etc.
 - Hotmail or MSN users automatically have Microsoft Passport accounts set up
- Passport may continue to evolve; bugs have been uncovered

Four parts of Passport account

- Passport Unique Identifier (PUID)
 - Assigned to the user when he or she sets up the account
- User profile, required to set up account
 - Phone number or Hotmail or MSN.com e-mail address
 - Also name, ZIP code, state, or country, ...
- Credential information
 - E-mail address or phone number
 - Minimum six-character password or PIN
 - Four-digit security key, used for a second level of authentication on sites requiring stronger sign-in credentials
- Wallet
 - Passport-based application at passport.com domain
 - E-commerce sites with Express Purchase function use wallet information rather than prompt the user to type in data

Passport log-in



Source: Microsoft

Sign-In of the Times. 1) The Passport authentication process begins when a user signs in from a participating Web site. 2) The user's browser is redirected to a Passport login server. 3) The Passport server prompts the user for an e-mail address and password and checks it against its database. 4) The Passport server places three encrypted cookies to the user's browser, which is redirected to the participating Web site. 5) The participating site decrypts the authentication cookie via the Passport Manager object. 6) The site serves up the requested Web page.

Trusted Intermediaries

Symmetric key problem:

- How do two entities establish shared secret key over network?

Solution:

- trusted key distribution center (KDC) acting as intermediary between entities

Public key problem:

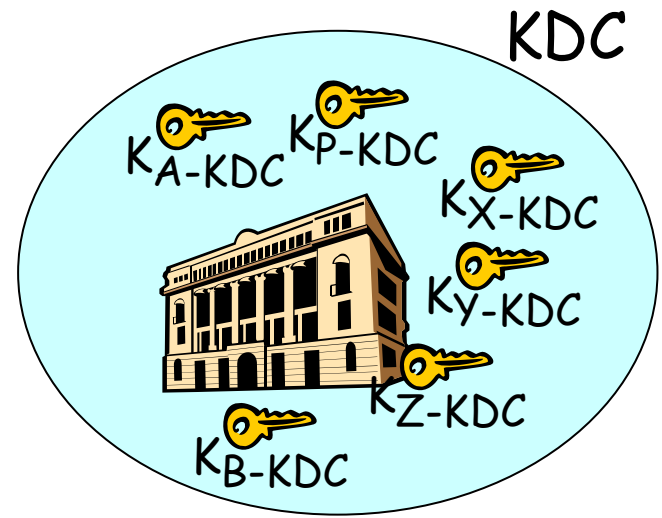
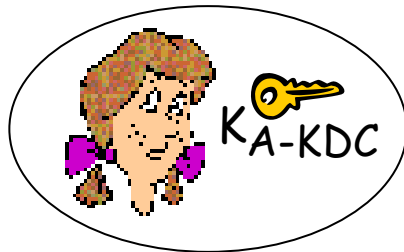
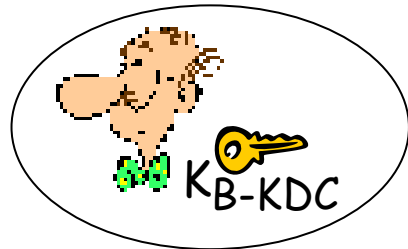
- When Alice obtains Bob's public key (from web site, e-mail, diskette), how does she know it is Bob's public key, not Trudy's?

Solution:

- trusted certification authority (CA)

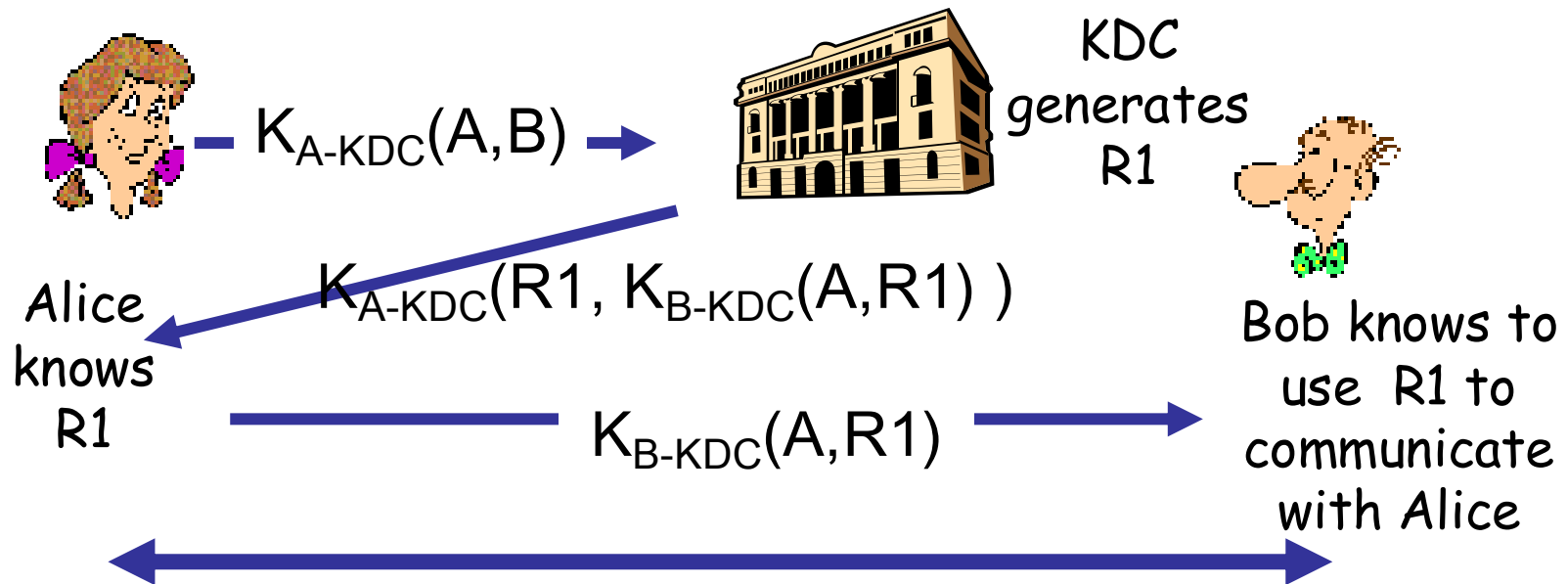
Key Distribution Center (KDC)

- Alice, Bob need shared symmetric key.
- **KDC**: server shares different secret key with each registered user (many users)
- Alice, Bob know own symmetric keys, K_{A-KDC} K_{B-KDC} , for communicating with KDC.



Key Distribution Center (KDC)

Q: How does KDC allow Bob, Alice to determine shared symmetric secret key to communicate with each other?



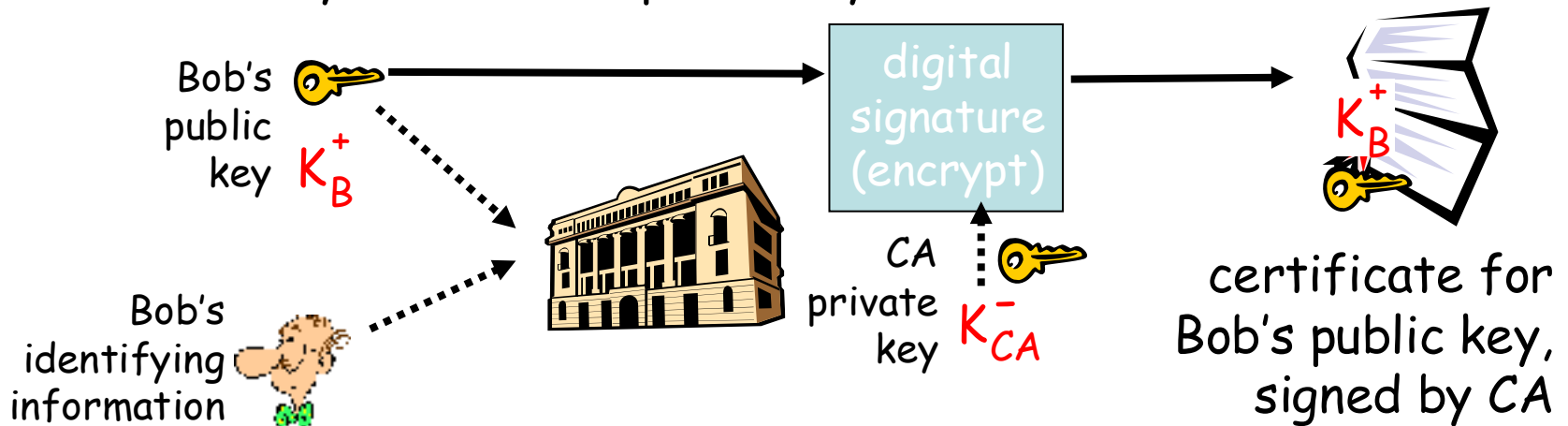
Alice and Bob communicate: using $R1$ as *session key* for shared symmetric encryption

Ticket

- In $K_{A-KDC}(R1, K_{B-KDC}(A,R1))$, the $K_{B-KDC}(A,R1)$ is also known as a ticket
- Comes with expiration time

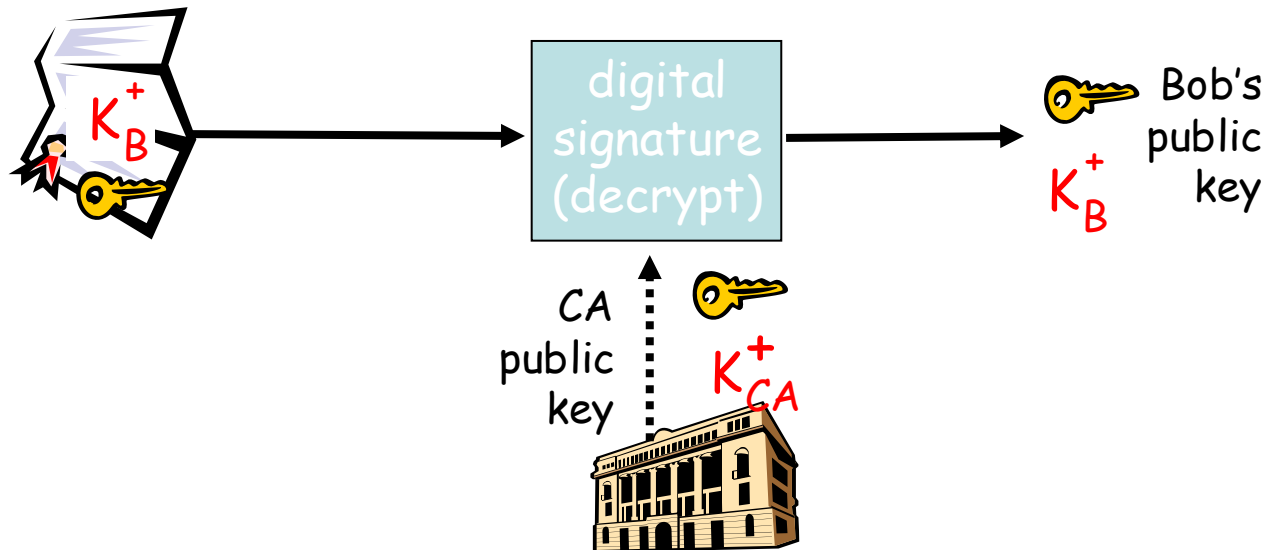
Certification Authorities

- **Certification authority (CA):** binds public key to particular entity, E.
- E (person, router) registers its public key with CA.
 - E provides "proof of identity" to CA.
 - CA creates certificate binding E to its public key.
 - Certificate containing E's public key digitally signed by CA
 - CA says "this is E's public key"



Certification Authorities

- When Alice wants Bob's public key:
 - gets Bob's certificate (Bob or elsewhere).
 - apply CA's public key to Bob's certificate, get Bob's public key



Single KDC/CA

- Problems
 - Single administration trusted by all principals
 - Single point of failure
 - Scalability
- Solutions: break into multiple domains
 - Each domain has a trusted administration

Multiple KDC/CA Domains

Secret keys:

- KDCs share pairwise key
- topology of KDC: tree with shortcuts

Public keys:

- cross-certification of CAs
- example: Alice with CA_A , Boris with CA_B
 - Alice gets CA_B 's certificate (public key p_1), signed by CA_A
 - Alice gets Boris' certificate (its public key p_2), signed by $CA_B(p_1)$

Advantages of salt

- Without salt
 - Same hash functions on all machines
 - Compute hash of all common strings once
 - Compare hash file with all known password files
- With salt
 - One password hashed 2^{12} different ways
 - Precompute hash file?
 - Need much larger file to cover all common strings
 - Dictionary attack on known password file
 - For each salt found in file, try all common strings